Spring Boot Guide 02: how the Waypoint backend works

This guide teaches Spring Boot from the ground up, then shows, with the real code, how the Waypoint backend uses it.

- **Part A** is the fundamentals any Spring developer needs.
- **Part B** answers "is this a layered architecture?" and "why a modular monolith?".
- **Part C** follows a request through the actual files.
- **Part D** covers the object-oriented design: every interface, abstract class, record and pattern, with counts.
- **Part E** maps every module, layer by layer.
- **Part F** covers the security logic.
- **Part G** lists the gaps and trade-offs.
- **Part H** is the practical checklists.

Written against `main` at `d086270` on 2026-10-16. Paths are under `backend/src/main/java/com/waypoint/dispatch/` unless they start elsewhere. Companion: AI and System Guide 01.

Contents

Part A. Spring Boot fundamentals

1. What Spring and Spring Boot are
2. Inversion of control and dependency injection
3. Beans: how Spring finds, builds and wires them
4. Starters and auto-configuration
5. Spring MVC: how an HTTP request is handled
6. Data access: JDBC, the pool, JdbcTemplate and transactions
7. Configuration
8. Scheduling, actuator and testing

Part B. The architecture 9. Layered, modular monolith or microservices 10. Why people choose Spring Boot, and why this project did

Part C. Our implementation, file by file 11. Start-up: `WaypointApplication` 12. The data layer: `DataConfig` and `Database` 13. A write, end to end: cancelling an order 14. A read, end to end 15. Events: the outbox and subscribers 16. Scheduled jobs 17. Errors: from exception to problem+json 18. Filters

Part D. Object-oriented design 19. The numbers 20. The four pillars, as used here 21. Every interface that matters 22. The five abstract classes 23. Records, enums and sealed types 24. Design patterns in the code 25. SOLID, checked against the code

Part E. Module map 26. Every module, layer by layer 27. The files to read first

Part F. Security logic 28. Security without the Spring Security filter chain

Part G. Gaps and trade-offs 29. What this design costs, and what to watch

Part H. Practice 30. Add a command, end to end 31. Add a read endpoint 32. Run, debug and test 33. Glossary

---

Part A. Spring Boot fundamentals

1. What Spring and Spring Boot are

Spring Framework is a Java library for building applications out of objects that it creates and connects for you. Its core is a container: you describe your objects and what each needs, and Spring builds them in the right order and hands each its dependencies. On top of that core sit modules for web servers (Spring MVC), databases (Spring JDBC, transactions), scheduling, validation and more.

Spring Boot is Spring with the setup done. Plain Spring needed you to declare the web server, the JSON mapper, the connection pool and dozens more pieces by hand. Spring Boot adds three things:

1. **Starters.** One dependency (`spring-boot-starter-web`) pulls in a compatible set of libraries (Tomcat, Spring MVC, Jackson).
2. **Auto-configuration.** On start it looks at what is on the classpath and what you configured, and creates sensible default objects for anything you did not define yourself.
3. **An embedded server and a runnable jar.** `java -jar app.jar` starts Tomcat inside the process. There is no separate server to install.

```
your code            @Component classes, @Configuration, application.properties
    |
Spring Boot          starters + auto-configuration + embedded Tomcat
    |
Spring Framework     IoC container, MVC, JDBC, transactions, scheduling
    |
Java 17 / JVM
```

In Waypoint: Spring Boot 3.4.1 on Java 17 (`backend/pom.xml`). It is one process, `WaypointApplication` , serving HTTP on port 8080.

2. Inversion of control and dependency injection

The problem. A class that builds its own dependencies is welded to them:

```
class CancelOrderHandler {
  private final JdbcOrderRepository orders = new JdbcOrderRepository(new Database(...)); // welded
}
```

You cannot test it without a real database, cannot swap the repository, and every class repeats the wiring.

Inversion of control (IoC). The class stops creating what it needs; something outside (the container) creates it and passes it in. **Dependency injection (DI)** is the passing in. Spring does it through the constructor:

```
@Component
public class CancelOrderHandler implements CommandHandler {
  private final JdbcOrderRepository orders;
  private final EventPublisher events;
  private final Metrics metrics;
  private final Clock clock;

  public CancelOrderHandler(JdbcOrderRepository orders, EventPublisher events, Metrics metrics, Clock clock) {
    this.orders = orders; this.events = events; this.metrics = metrics; this.clock = clock;
  }
}
```

This is `ordering/application/CancelOrderHandler.java`, exactly as in the code. Spring sees one constructor, finds an object of each parameter type, and calls it.

```
          Spring container (ApplicationContext)
   +-----------------------------------------------------+
   |  Clock ------------------+                          |
   |  Metrics ----------------+--> CancelOrderHandler     |
   |  EventPublisher ----------+          |               |
   |  JdbcOrderRepository -----+          v               |
   |        ^                        CommandBus          |
   |     Database <-- JdbcTemplate <-- DataSource (pool) |
   +-----------------------------------------------------+
   Spring builds from the bottom up, then injects upwards.
```

Why constructor injection.

- The fields can be `final`, so an object is complete the moment it exists.
- A test calls the constructor with fakes; no Spring is needed.
- A missing dependency fails at start-up, not at the first request.

Waypoint uses constructor injection only. **There is not one `@Autowired` in the main code** (counted: 0), because with a single constructor Spring needs no annotation.

Injecting many, or maybe one. Two forms Waypoint relies on:

Parameter type	Spring injects	Used in Waypoint for
`List<CommandHandler>`	**every** bean that implements the interface	the command bus collects all 72 handlers
`List<ScheduledJob>`	every job	the scheduler runs all 39 jobs
`Optional<CommandAuthorizer>`	the bean if one exists, else empty	the bus fails closed when there is none
`ObjectProvider<T>`	a lazy handle to an optional bean	`TimeConfig` asks for the demo time adjustment

This is how a new command is added with no registration code: write a class that implements `CommandHandler`, mark it `@Component`, and it is in the list.

3. Beans: how Spring finds, builds and wires them

A **bean** is an object the container created and manages. There are two ways to make one.

1. Component scanning. `@SpringBootApplication` on `WaypointApplication` tells Spring to scan its package (`com.waypoint.dispatch`) and everything below it, and to make a bean of every class carrying a stereotype annotation:

Annotation	Meaning	Count in Waypoint
`@Component`	a general bean	362
`@RestController`	a bean that answers HTTP; return values become JSON	40
`@Configuration`	a class whose `@Bean` methods build beans by hand	28
`@ConfigurationProperties`	a typed, validated view of configuration	18 (found by `@ConfigurationPropertiesScan`)

2. `@Bean` **methods** (34 in Waypoint), for objects you build yourself, often from a library:

```
@Configuration
public class DataConfig {
  @Bean
  public JdbcTemplate jdbcTemplate(DataSource dataSource) {
    JdbcTemplate template = new JdbcTemplate(dataSource);
    template.setQueryTimeout(15);
    return template;
  }
}
```

(`platform/config/DataConfig.java` .) The parameter `DataSource` is itself a bean, built by another `@Bean` method, so Spring resolves the order.

Lifecycle.

```
  scan + read @Bean methods  ->  build the dependency graph  ->  instantiate in order
      ->  inject  ->  callbacks (SmartInitializingSingleton.afterSingletonsInstantiated)
      ->  ApplicationRunner.run  ->  serving  ->  shutdown (DisposableBean.destroy)
```

Waypoint uses those hooks:

- `ScheduledJobRunner` implements `SmartInitializingSingleton` , to start the cron timers once every bean exists, and `DisposableBean` , to stop the threads.
- `WaypointApplication` implements `ApplicationRunner` , to run CLI commands such as `migrate` .

Scope. Every Waypoint bean is a **singleton**: one instance for the whole process, shared by every request thread. That makes thread safety a rule: beans hold only `final` collaborators and keep request data in local variables. The few pieces of mutable state are guarded:

- `CircuitBreaker` methods are `synchronized` ;
- the scheduler's bulkheads are a `ConcurrentHashMap` ;
- `Database` keeps the current actor in a `ThreadLocal` .

Conditional beans. `@ConditionalOnProperty` creates a bean only when a setting allows it:

```
@ConditionalOnProperty(name = "app.scheduling.enabled", havingValue = "true", matchIfMissing = true)
public class ScheduledJobRunner ...
```

Tests set `app.scheduling.enabled=false` and `app.relay.enabled=false` , so no timer fires mid-test. The proof store picks its adapter the same way: `app.execution.proof-store=database` or `local` .

4. Starters and auto-configuration

What each dependency in `backend/pom.xml` brings, and what Waypoint does with it:

Dependency	Gives	Waypoint's use
`spring-boot-starter-web`	Tomcat, Spring MVC, Jackson	all 161 HTTP endpoints, JSON in and out
`spring-boot-starter-jdbc`	DataSource, JdbcTemplate, transaction manager	used, but **every one is redefined** in `DataConfig` (section 12)
`spring-boot-starter-validation`	Jakarta Bean Validation	`@Min`, `@NotNull` and `@AssertTrue` on configuration records
`spring-boot-starter-actuator`	health, metrics endpoints	`/health/liveness`, `/health/readiness`, `/prometheus`
`micrometer-registry-prometheus`, tracing, OTLP	metrics and traces	behind `platform/observability/Metrics`
`spring-security-crypto`	**only** the password encoders	`Argon2PasswordEncoder`; the Spring Security web filter chain is **not** used (section 28)
`bcprov-jdk18on`	Bouncy Castle	Argon2 and web-push cryptography
`postgresql`	the JDBC driver	
`commons-csv`, `jackson-databind`	CSV and JSON	reference data import
`ortools-java`	Google OR-Tools	planning benchmarks and solvers
test: `spring-boot-starter-test`, JUnit 5, ArchUnit, PostgreSQL	testing	section 8

How auto-configuration decides. Each auto-configuration class is guarded by conditions such as "a DataSource class is on the classpath" and "the application has not defined its own". Define your own bean of that type and Spring's backs off. Waypoint does this for the data layer, because it needs things the default cannot give:

- every connection set to an unprivileged role;
- serializable transactions;
- a startup that does not block on the database.

What is not here. There is no JPA or Hibernate (`spring-boot-starter-data-jpa` is absent). SQL is written by hand and run with `JdbcTemplate`. Section 6 explains why.

5. Spring MVC: how an HTTP request is handled

```
HTTP request
    |
Tomcat (embedded, port 8080)
    |
Servlet filters, in @Order:  CorrelationId -> RequestSize -> OriginGuard
    |                        -> McpCredential -> ConditionalReads, plus SecurityHeaders
    v
DispatcherServlet  (Spring MVC's front controller)
    |
HandlerMapping: finds the controller method for path + verb
    |               (@RestController + @GetMapping("/day"))
    v
argument resolution: @RequestParam, @PathVariable, @RequestBody (Jackson: JSON -> record)
    |
your controller method  ->  returns a record / List / Map
    |
HttpMessageConverter (Jackson): object -> JSON
    |
on exception: @ExceptionHandler in @RestControllerAdvice -> problem+json
    v
HTTP response
```

Filters vs controllers. A filter (`OncePerRequestFilter`) sees every request before Spring MVC routes it. It is the place for cross-cutting work: request ids, size limits, origin checks, headers. A controller handles one route. Waypoint keeps controllers thin: resolve the caller, check permission, call a query or the bus, return.

A real controller (`ordering/web/OrderController.java`):

```java
@RestController
@RequestMapping("/api/orders")
public class OrderController {
  @GetMapping("/day")
  public List<OrderView> ordersForDay(
      @RequestParam String depot, @RequestParam LocalDate date, HttpServletRequest request) {
    var actor = authorizer.require(request, READ, "wpt:order:depot:" + depot);
    return orders.ordersForDay(actor, depot, date);
  }
}
```

Spring parses `date` into a `LocalDate`, calls the method, and Jackson writes the list of `OrderView` records as a JSON array.

One endpoint for all writes. Writes do not get a route each. They go to `POST /api/commands` (`platform/web/CommandController.java`), which is thin by contract:

```java
@PostMapping
public CommandAck submit(@RequestBody CommandRequest body, HttpServletRequest request) {
  Command command = toCommand(body);
  Actor actor = actors.flatMap(r -> r.resolveCommand(request, command))
      .orElseThrow(() -> new DomainException(ErrorCode.UNAUTHENTICATED, "Not signed in"));
  CommandResult result = bus.dispatch(actor, command, (String) request.getAttribute(CorrelationIdFilter.ATTRIBUTE));
  return new CommandAck(command.commandId().toString(), command.kind(), result.replayed(), result.value());
}
```

Why one door. An offline phone queues the envelope and replays it verbatim hours later. The same envelope, the same idempotency key and the same answer work only if there is one shape for every write.

6. Data access: JDBC, the pool, JdbcTemplate and transactions

6.1 The stack, bottom up

```
PostgreSQL 16
    ^   TCP, the PostgreSQL wire protocol
JDBC driver (org.postgresql)        Java's standard database API
    ^
HikariCP connection pool            8 open connections, reused (DataSource)
    ^
DataSourceTransactionManager        begins, commits, rolls back on a connection
    ^
TransactionTemplate                 "run this code in a transaction" with settings
    ^
JdbcTemplate                        run SQL, bind parameters, map rows, close resources
    ^
Database (Waypoint's seam)          role + actor + retry + read/write variants
    ^
Jdbc*Repository (per module)        the module's SQL
```

Pool. Opening a database connection takes milliseconds; a pool keeps a few open and lends them out. HikariCP is Spring Boot's default and the fastest common pool. Waypoint sets:

- `maximumPoolSize = 8` (`DB_POOL_SIZE`);
- `connectionTimeout = 3 s`, so a dead database fails fast;
- `initializationFailTimeout = -1`, so the process starts even when PostgreSQL is down;
- `connectionInitSql = "SET ROLE waypoint_app"`, so every connection starts unprivileged.

JdbcTemplate. It removes JDBC boilerplate: getting a connection, preparing the statement, binding `?` parameters (which prevents SQL injection), reading rows and closing everything even on error.

```
jdbc.queryForList("SELECT * FROM ordering.orders WHERE depot_code = ? AND delivery_date = ?", depot, date);
jdbc.update("UPDATE ... WHERE order_id = ? AND row_version = ?", ..., id, expected);
```

6.2 Transactions

A transaction makes a group of statements all-or-nothing. Spring offers two ways:

Way	How	Waypoint
Declarative	`@Transactional` on a method; a proxy opens and commits around the call	**not used** (counted: 0)
Programmatic	`TransactionTemplate.execute(status -> work)`	**used everywhere**, inside `Database`

Why programmatic here. Every Waypoint transaction must run `SET LOCAL ROLE` and `SET LOCAL app.actor_id` first, choose its isolation, retry a serialization failure, and sometimes suspend the caller's transaction. One method (`Database.asModule`) does all of that, and an ArchUnit test makes sure nothing else touches JDBC (section 8). `@Transactional` would also fail silently on a call from inside the same class, where Spring's proxy is bypassed; the programmatic form has no such trap.

Settings that matter.

Setting	Values	Waypoint
Isolation	READ COMMITTED, REPEATABLE READ, SERIALIZABLE	**SERIALIZABLE** for commands. Two commands that would conflict cannot both commit; one gets SQLSTATE `40001` and is retried
Propagation	REQUIRED (join), REQUIRES_NEW (suspend and start a new one)	REQUIRED for commands; REQUIRES_NEW for contract reads (`readAs`) and for records that must survive a rollback (`asSystemSeparately`)
Read-only	true or false	true for `readAs`
Timeout	seconds	15 s for the transaction and each query

6.3 Why not JPA or Hibernate

JPA maps classes to tables and writes the SQL for you. It is the common choice, and a fine one for create, read, update and delete screens. Waypoint chose hand-written SQL because:

- **Row-level security** needs a `SET LOCAL` at the start of each transaction, under Waypoint's control.

- **The version guard must be visible.** `UPDATE ... WHERE row_version = ?` checked for zero rows is the concurrency control, and with JPA it would be hidden.
- **The domain must stay pure** (rule 2): JPA entities carry annotations and lazy-loading proxies into the domain.
- **The heavy reads are SQL work:** planning and reporting queries use window functions, `FOR UPDATE SKIP LOCKED`, `jsonb` and partial indexes.
- **No surprise queries,** such as the N+1 problem or lazy loading outside a transaction.

The cost is more code (every row is mapped by hand), covered in Part G.

7. Configuration

Where values come from. `src/main/resources/application.properties`, with each value usually read from an environment variable with a default:

```
app.session.absolute-lifetime=${SESSION_ABSOLUTE_LIFETIME:12h}
app.ai.enabled=${AI_ENABLED:false}
spring.datasource.hikari.maximum-pool-size=${DB_POOL_SIZE:8}
```

`${NAME:default}` means: the environment variable `NAME`, else `default`. A deployment changes behaviour through `.env` and `compose.yaml` with no rebuild.

Typed and validated. Each group is bound to a Java record:

```
@Validated
@ConfigurationProperties(prefix = "app.ai")
public record AiProperties(
    @DefaultValue("false") boolean enabled,
    @DefaultValue("groq") String provider,
    @DefaultValue("300") @Min(0) @Max(100_000) int dailyCalls, ... ) {

  @AssertTrue(message = "app.ai.enabled needs the provider's key (GROQ_API_KEY or OPENROUTER_API_KEY)")
  public boolean isKeyPresentWhenEnabled() { return !enabled || (key() != null && !key().isBlank()); }
}
```

(`platform/config/AiProperties.java`.) A bad value stops the process at start-up with a message naming the setting. The rule is: refuse to start rather than run misconfigured. The 18 properties records cover sessions, AI, transcription, MCP, the object store, retention, demo sign-in and more.

Secrets (keys, passwords) come only from the environment, are never logged, and some records override `toString()` to leave them out (`TranscribeProperties`).

8. Scheduling, actuator and testing

Scheduling. Spring has `@Scheduled`. Waypoint deliberately does not use it; it has its own `ScheduledJobRunner` over Spring's `ThreadPoolTaskScheduler` (section 16). The reasons:

- a database lease per job, so two replicas never both run the 16:00 cutoff;
- a business clock that demo mode can move;
- a thread per external dependency;
- a record of every run.

Actuator. These settings expose `/health/liveness`, `/health/readiness` and `/prometheus`:

```
management.endpoints.web.exposure.include=health,prometheus,info
management.endpoints.web.base-path=/
```

Readiness includes the database; liveness does not. A database outage then makes the instance "not ready" (no traffic) instead of "dead" (restarted in a loop).

Testing (`backend/src/test/java`, 178 test classes):

Kind	Tool	Example
Domain unit	plain JUnit 5, no Spring	`IssueTriageTest`, policy evaluator tests
Integration	`@SpringBootTest` with a real PostgreSQL	`OrderingSchemaIntegrationTest`, `DemoSignInIntegrationTest`
Architecture	ArchUnit	`ModuleBoundaryTest`, `EventCatalogueTest`

An integration test starts the whole application context against a test database:

```
@SpringBootTest
@ExtendWith(TestDatabase.class)
class DemoSignInIntegrationTest {
  @Autowired Migrator migrator;     // field injection is fine in tests
  @DynamicPropertySource
  static void database(DynamicPropertyRegistry registry) {
    registry.add("app.database-url", TestDatabase::url);
  }
}
```

`TestDatabase` picks `TEST_DATABASE_URL`, else a throwaway PostgreSQL container, else skips with the reason. Spring caches the context between test classes with the same configuration, which keeps the suite fast.

ArchUnit reads the compiled classes and fails the build when a rule breaks. `ModuleBoundaryTest` has these tests:

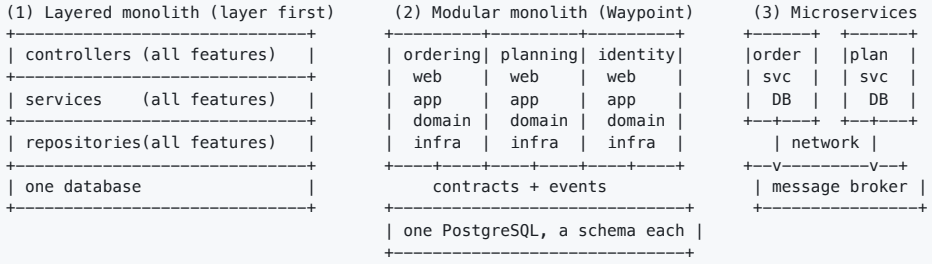
- `sharedKernelDependsOnNothingInternal`;
- `platformCarriesNoBusinessModule`;
- `modulesReachEachOtherOnlyThroughContracts`;
- `domainPackagesStayFrameworkFree`;
- `contractPackagesStayFrameworkFree`;
- `eventsAreRecordsInAContract`;
- `onlyThePlatformSeamTouchesJdbcDirectly`;
- `onlyInfrastructureAndApplicationTouchTheDatabaseSeam`;
- `noClassIsNamedService`.

Part B. The architecture

9. Layered, modular monolith or microservices

Short answer. Both layered and modular. At the top level it is a **modular monolith**: one deployable, split into 16 business modules that may talk only through published contracts. **Inside each module it is layered**, in the ports and adapters style: `contract`, `domain`, `application`, `infrastructure` and `web`.

9.1 The three shapes



	Layered monolith	Modular monolith	Microservices
Packages	by layer (`controllers/`, `services/`)	by business capability, layers inside	one repository or more per service
Deploy	one	one	many
Change several things atomically	yes, any table	yes, one transaction, but through contracts	no; needs sagas and compensation
Boundaries enforced by	nothing; anything can call anything	tests (ArchUnit) and database roles	the network
Typical failure	one 1,780-line class that does everything	a module reaching around its contract (tests stop it)	distributed failures, versioning, ops cost
Team size it suits	small, short-lived apps	**one team to a few, a growing domain**	many teams, independent scaling

The prototype this replaced was layer-first, and one class grew to 1,780 lines (AGENTS.md rule 1). That is the failure the modular shape prevents.

9.2 Why a modular monolith for this problem

1. **Many decisions must commit together.** Publishing a plan changes the plan, writes the receipt and the audit row, and queues events for four modules. In one process that is one database transaction. As microservices it would be a saga, with compensation for every partial failure.
2. **One small team, one VPS.** One deployable, one database, one log stream. Microservices would multiply the infrastructure the team must run (roadmap 03: no RabbitMQ, no Kafka, no Redis yet).
3. **Boundaries without the network.**
   - The modules talk only through contracts and outbox events, exactly as services would.
   - Each module owns a schema and a database role.
   - There are no foreign keys between modules.
   - A module can therefore be extracted into a service later by turning its contract into an HTTP API and its outbox into a broker. ADR-001 names the triggers for doing that.
4. **The heavy work is separate where it should be.**
   - **Python models:** a different language and their own failure modes, so they are a separate process behind a port.
   - **Speech:** the same, plus a GPL licence, so it is a separate process too.

9.3 The layers inside one module

```
          other modules
               |  (only through contract/)
   +-------------v---------------+
   | contract/   views, commands, events, query interfaces   (public API of the module)
   +-----------------------------+
   | web/        @RestController: parse, authorize, call      (HTTP in)
   +-----------------------------+
   | application/ handlers, queries, jobs, consumers          (transactions, authorization)
   +-----------------------------+
   | domain/     pure rules, state machines, value objects    (no Spring, no SQL, no clock)
   +-----------------------------+
   | infrastructure/ Jdbc*Repository, HTTP adapters           (SQL out, ports out)
   +-----------------------------+
               |
          PostgreSQL / external APIs
```

The dependency direction:

```
web -> application -> domain <- infrastructure
          |                          ^
          +---- uses ports ----------+
```

The domain depends on nothing. Infrastructure implements interfaces the application or domain declares. This is the **dependency inversion** that makes `domain` testable with no database.

10. Why people choose Spring Boot, and why this project did

Why the industry picks it.

- **Maturity and ecosystem.** Twenty years old, with a library for almost anything and long-term support releases.
- **Productivity.** Starters and auto-configuration give a running web service in minutes.
- **Production features built in:** health checks, metrics, tracing and configuration binding.
- **Java's strengths:**
  - static types catch mistakes before they run;
  - the JVM is fast, and its garbage collection and threading are well proven;
  - the tooling (IDEs, debuggers, profilers) is mature.
- **People.** Many developers know it, and its patterns are documented everywhere.

Why it fits Waypoint.

- **The rules are complex and must be exact:**

  - planning constraints;
  - permission policies;
  - state machines;
  - versioned writes.

  A typed language with pure, unit-tested domain classes suits that.

- **The work is transactional.** Spring's transaction support and JDBC give full control of PostgreSQL's serializable isolation and RLS.

- **There is a lot of in-process background work:** the outbox relay, 39 scheduled jobs and circuit breakers. That is simple in a long-running JVM process.

- **Planning needs optimisation libraries.** Google OR-Tools has a Java binding.

The alternatives, and why not:

- **Node with NestJS:** weaker static typing for the domain.
- **Django:** an ORM-first design, and Python in the core.
- **Go:** fewer libraries for this kind of enterprise plumbing.

Each could work; Spring Boot fits best with a team that knows Java and a rules-heavy, transactional domain.

Part C. Our implementation, file by file

11. Start-up: `WaypointApplication`

`WaypointApplication.java` is the entry point.

```
@SpringBootApplication           // scan com.waypoint.dispatch, auto-configure
@ConfigurationPropertiesScan     // bind every @ConfigurationProperties record
public class WaypointApplication implements ApplicationRunner { ... }
```

Serving vs commands. With no arguments, it serves HTTP. With command arguments, it runs them and exits, never serving:

```
java -jar app.jar migrate import-reference demo-accounts seed-delivery-day
```

The commands are `migrate`, `import-reference`, `account-create`, `account-grant-depot`, `demo-accounts`, `operator-pin`, `loading-fixture`, `seed-delivery-day` and `forecast-run`. This is the rule "builds and requests never migrate, import or seed": those happen only when an operator runs them explicitly (`scripts/compose-init.sh` does it once on install).

12. The data layer: `DataConfig` and `Database`

`platform/config/DataConfig.java` replaces Spring Boot's defaults with four beans:

Bean	Setting	Why
`DataSource` (Hikari)	`connectionInitSql = "SET ROLE waypoint_app"`	every connection starts with no privileges; a transaction that forgets its module role fails
`JdbcTemplate`	`setQueryTimeout(15)`	no runaway query
`PlatformTransactionManager`	`DataSourceTransactionManager`	plain JDBC transactions
`TransactionTemplate serializableTransactions`	`ISOLATION_SERIALIZABLE`, 15 s timeout	the strongest isolation by default; anything weaker is an argued exception

`platform/db/Database.java` is the only door to PostgreSQL. Every module calls one of these:

Method	Transaction	Runs as	Use
`asModule(role, actorId, work)`	serializable, joins the bus's	module role, the actor	a command's work
`asSystem(role, work)`	serializable	module role, `SYSTEM_ID`	jobs, event consumers
`readAs(role, actorId, work)`	**new**, read committed, read-only	module role, the actor	a contract query called from another module
`asSystemSeparately(role, work)`	**new**, read committed	module role, system	a record that must survive the caller's rollback
`afterCommit(runnable)`	after the commit		clear a cache only once the change is real
`updateExpectingOneRow(sql, ...)`	inside the current transaction		the optimistic version guard

What happens inside `asModule` (the real code, shortened):

```
public <T> T asModule(ModuleRole role, UUID actorId, Supplier<T> work) {
  for (int attempt = 0; ; attempt++) {
    try {
      return serializable.execute(status -> runAs(role, actorId, work));
    } catch (RuntimeException e) {
      String state = sqlState(e);
      if ("42501".equals(state)) metrics.increment("waypoint.db.permission_denied", "module", module);
      boolean retryable = "40001".equals(state) || "40P01".equals(state);   // serialization, deadlock
      if (retryable && attempt < MAX_ATTEMPTS - 1) { sleep(20L * (attempt + 1)); continue; }
      throw e;
    }
  }
}

private <T> T runAs(ModuleRole role, UUID actorId, Supplier<T> work) {
  jdbc.execute("SET LOCAL ROLE " + role.roleName());                      // which tables
  if (actorId != null)
    jdbc.queryForList("SELECT set_config('app.actor_id', ?, true)", actorId.toString()); // which rows
  AMBIENT_ACTOR.set(actorId);
  try { return work.get(); } finally { AMBIENT_ACTOR.remove(); }
}
```

Five details worth learning:

1. `SET LOCAL`**, not** `SET`**.** The setting ends with the transaction. A pooled connection would otherwise carry one person's identity into the next request: a classic security bug.
2. **The actor is a bound parameter** (`?`), never concatenated, so it cannot be used for SQL injection. The role name comes from an enum, never from input.
3. **Retry on** `40001` **and** `40P01`**.** Under serializable isolation PostgreSQL may abort one of two conflicting transactions, and the whole unit of work runs again, validation included.
4. `ModuleRole` **is an enum** of 15 roles. Its value is the only way to name a database role, so a typo cannot compile.
5. `ThreadLocal` **ambient actor.** A contract query called deep inside a command can learn who is acting without every method passing it.

13. A write, end to end: cancelling an order

```
browser                   POST /api/commands {commandId, kind:"order:Cancel", expectedVersion:4,
  |                                           payload:{orderId, reason}}
  v
Next.js proxy  (frontend/app/api/[...path]/route.ts)  -> backend:8080
  v
Filters: CorrelationId -> RequestSize -> OriginGuard -> ...
  v
CommandController.submit          parse envelope, ActorResolver -> Actor   (platform/web)
  v
CommandBus.dispatch                                                        (platform/messaging)
  1 find handler for "order:Cancel"         unknown kind -> 403 + audit
  2 CommandAuthorizer.denyReason            PolicyDecisionPoint: policy for actor (identity)
  3 open serializable tx as waypoint_ordering, actor set     (Database.asModule)
  4 denyReasonInTransaction                  re-check inside the tx
  5 IdempotencyGuard                         seen commandId? same fingerprint -> replay
  6 handler.handle(actor, command)  ------------------------------+
  7 commit: state + receipt + audit row + outbox event             |
                                                                   v
CancelOrderHandler.handle                                          (ordering/application)
  order = repository.current(orderId, expectedVersion)
  cancelled = order.cancel()            -> OrderStateMachine allows the edge? (ordering/domain)
  orders.update(cancelled, expected)    -> UPDATE ... WHERE row_version = ?  (infrastructure)
  orders.recordStatus(...)              -> status history row
  events.publish(actor, OrderCancelled) -> integration.outbox_events, same tx
  return {orderId, status, rowVersion}
```

The handler (`ordering/application/CancelOrderHandler.java`). It declares four things to the bus and does the work:

```java
@Override public String kind()   { return OrderCommands.CANCEL; }        // routing key
@Override public String action() { return OrderCommands.CANCEL; }        // policy action
@Override public ModuleRole moduleRole() { return ModuleRole.ORDERING; } // database role
@Override public String resource(Command command) {                      // policy resource
  UUID id = CommandPayload.of(command).optionalUuid("orderId");
  return id == null ? null : "wpt:order:order:" + id;
}
```

The domain decides the move (`ordering/domain/OrderStateMachine.java`):

```java
EDGES.put(CONFIRMED, EnumSet.of(ALLOCATED, DEFERRED, UNSERVABLE, CANCELLED));
EDGES.put(LOADING,   EnumSet.of(IN_TRANSIT, DEFERRED));      // no CANCELLED: goods are on a truck
EDGES.put(FAILED,    EnumSet.noneOf(OrderStatus.class));     // terminal
```

`Order.cancel()` refuses an illegal move with a `DomainException`, so the rule lives in one pure class, testable without Spring. A SQL `CHECK` only restricts values, not moves.

The repository guards the version (`ordering/infrastructure/JdbcOrderRepository.java`):

```java
database.updateExpectingOneRow("""
    UPDATE ordering.orders
       SET status = ?, ..., row_version = row_version + 1, updated_at = ?
     WHERE order_id = ? AND row_version = ?
    """, ..., next.orderId(), expectedVersion);
```

If someone changed the order since the client read version 4, zero rows match and the method throws `VERSION_CONFLICT` (`409`). The client re-reads; nothing is overwritten.

Why the warehouse is not called here. The event carries the reservation reference; the warehouse module cancels it when it consumes `order.cancelled`. The cancel therefore commits even when the warehouse is down, and is completed when it returns.

14. A read, end to end

```
GET /api/orders/day?depot=Peliyagoda&date=2026-10-17
  v
OrderController.ordersForDay
  actor = authorizer.require(request, "order:Read", "wpt:order:depot:Peliyagoda")
      SessionRequestAuthorizer: cookie -> session -> Actor
                                PolicyDecisionPoint: allowed? else 403 + audit
  v
OrderDataQuery.ordersForDay(actor, depot, date)
  requireScope(...)  "SELECT app.actor_has_depot(?)"  -> outside scope: 403 + audit, never []
  database.readAs(ORDERING, actor, () -> repository.forDay(depot, date))
      SET LOCAL ROLE waypoint_ordering; set_config('app.actor_id', ...)
      SELECT ... FROM ordering.orders WHERE depot_code = ? AND delivery_date = ?
      PostgreSQL row-level security keeps only rows the actor may see
  v
List<OrderView> -> Jackson -> JSON
```

There are three checks, at three levels:

1. the policy says the action is allowed;
2. the scope check makes a list outside scope a loud `403`;
3. RLS filters every row in SQL, so even a bug in Java cannot leak another depot's orders.

15. Events: the outbox and subscribers

Publishing (`platform/messaging/EventPublisher.java` , interface). The handler calls `events.publish(actor, event)` . The implementation, `OutboxEventPublisher` , inserts the event into `integration.outbox_events` **in the same transaction**. If the command rolls back, the event never existed.

Delivering (`OutboxRelay` , run by `OutboxRelayWorker`):

```
every pass:
  claim due rows  SELECT ... FOR UPDATE SKIP LOCKED  + lease   (replicas share the work)
  for each event, for each EventSubscriber<E> of its type:
      new transaction as subscriber.moduleRole(), actor = SYSTEM
      inbox claim (consumerName, eventId)  -> already applied? skip
      subscriber.on(envelope)
      commit
  failure -> backoff and retry; after N failures -> dead letter (replay: platform:ReplayEvent)
```

Subscribing. Implement `EventSubscriber<E>` and annotate with `@Component` ; Spring's `List<EventSubscriber<?>>` injection finds it. Here is the Jev shadow subscriber's shape (`intelligence/application/JevShadow.java`):

```
@Component
static class OnIssueRaised implements EventSubscriber<IssueRaised> {
  public String consumerName() { return "ml.on-issue-raised-jev"; }     // never rename: it is the inbox key
  public Class<IssueRaised> eventType() { return IssueRaised.class; }
  public ModuleRole moduleRole() { return ModuleRole.ML; }
  public void on(EventEnvelope<IssueRaised> envelope) { ... }          // must be safe to run twice
}
```

There are 23 classes that implement `EventSubscriber` directly. Many more extend a module's abstract consumer: `NotificationConsumer` 29 times, `OrderingConsumer` 13, `PlanningConsumer` 9, `IssuesConsumer` 7.

16. Scheduled jobs

The interface (`platform/scheduling/ScheduledJob.java`):

```
public interface ScheduledJob {
  String name();                // unique; also the advisory-lock key
  String cron();                // Spring cron, Asia/Colombo
  ModuleRole moduleRole();
  void run(Instant now);        // the clock is passed in: testable, demo-movable
  default boolean onBusinessClock() { return false; }
  default String bulkhead() { return null; }   // "ml", "warehouse", "push": its own thread
}
```

The runner (`ScheduledJobRunner`). It receives `List<ScheduledJob>` (39 jobs) and, for each one:

1. schedules it on a `ThreadPoolTaskScheduler` ;
2. before each run, takes a PostgreSQL **session advisory lock** named after the job on a separate connection, and skips the run if another replica holds it;
3. records the run in `integration.job_runs` ;
4. runs a job that names a bulkhead (an external dependency) on that dependency's single thread, so a model call that takes minutes cannot delay the 16:00 cutoff;
5. logs and counts a failing run, never rethrowing it, so one bad run does not unschedule the job.

Examples:

Job	Cron
`ml.attention-watch`	every minute
`ml.jev-shadow`	`30 * * * * *`
the voice transcripts	every 30 s
the order cutoff	16:00 on the business clock
retention	nightly

17. Errors: from exception to problem+json

One exception type for broken rules (`shared/error/DomainException.java`):

```
public class DomainException extends RuntimeException {
  private final ErrorCode code;              // VALIDATION_FAILED, NOT_FOUND, CONFLICT, VERSION_CONFLICT,
  private final List<Violation> violations;  // FORBIDDEN, UNAUTHENTICATED, RATE_LIMITED, ...
  ...
}
```

One translator (`platform/web/ApiExceptionHandler.java` , `@RestControllerAdvice`) has an `@ExceptionHandler` per exception type:

- `DomainException` goes to its HTTP status;
- unreadable JSON, a wrong argument type or a missing parameter is `400` or `422` ;
- `CannotCreateTransactionException` (the database is down) is `503` ;
- a `DataIntegrityViolationException` is a conflict.

Every answer is RFC 9457 `application/problem+json` :

```
{ "type": "about:blank", "title": "Conflict", "status": 409, "code": "VERSION_CONFLICT",
  "detail": "Row version did not match; the record changed", "violations": [] }
```

A detail never echoes framework or driver text (which can carry SQL or class names), and never personal data.

18. Filters

Each filter extends `OncePerRequestFilter` and runs in `@Order`:

Order	Filter	Does
+2	`CorrelationIdFilter`	a request id, carried to logs and audit rows
+3	`RequestSizeFilter`	refuses an oversized body early
+4	`OriginGuardFilter`	a state-changing request naming a foreign `Origin` is refused
+5	`McpCredentialFilter` (identity)	recognises an MCP bearer credential and marks the request
+6	`ConditionalReadsFilter`	ETag and `304` on the polled reads listed in `CacheablePaths`
default	`SecurityHeadersFilter`	`X-Frame-Options: DENY` and the other response headers

Part D. Object-oriented design

19. The numbers

Counted over `backend/src/main/java` (673 files, 78,042 lines):

Construct	Count	Note
Classes (including nested)	683	229 of them `final`
Records	653	immutable data: views, commands, events, value objects
Interfaces	53	5 of them `sealed`
Abstract classes	5	all template bases for handlers or consumers
Enums	77	statuses, error codes, `ModuleRole`, `Effect`
`extends` (class inheritance)	125	mostly of the 5 abstract bases and `OncePerRequestFilter`
`CommandHandler` implementations	72 classes, many holding several handlers	one per command kind
`EventSubscriber` implementations	23 direct, about 60 through abstract bases	
`ScheduledJob` implementations	39	
HTTP endpoint methods	161	in 40 controllers
Spring beans	362 `@Component`, 34 `@Bean`, 18 properties records	all singletons
`Optional<...>` uses	1,208	absence is a type, not `null`
Pattern-matching `instanceof`	60	Java 17 `if (x instanceof Foo f)`
`@Autowired`, `@Transactional`	0, 0	constructor injection; programmatic transactions

A Java object is created per bean once at start-up (about 400 singletons), and per request for the records and domain objects it handles. There is no object pool or cache of domain objects: they are short-lived and immutable, which the JVM's garbage collector handles cheaply.

20. The four pillars, as used here

Encapsulation: hide the state, expose behaviour.

- Fields are `private final`, set once in the constructor.
- `Order` is a record whose state changes only by returning a **new** `Order` from a method such as `order.cancel()`, `order.deferTo(date, n)` or `order.allocateTo(trip)`. Each method checks the state machine first, so no caller can put an order into an illegal status.
- `Database` hides JDBC, the role and the actor behind six methods. The `JdbcTemplate` field is never handed out.

Abstraction: depend on what, not how.

- `LanguageModel` says "complete this instruction with these facts, or answer nothing". Whether that is Groq or OpenRouter is the adapter's business (`OpenAiCompatibleModel`).
- `DecisionPort` says "choose one of these options", and Jev is behind it (`JevDecisions`).
- `RequestAuthorizer` says "check this action on this resource for this request". The session and policy code is in identity.

Inheritance, used sparingly. There are only 5 abstract classes, each a template for a family:

```
abstract class OrderingConsumer<E extends DomainEvent> implements EventSubscriber<E> {
  protected final OrderTransitions transitions;
  OrderingConsumer(OrderTransitions transitions) { this.transitions = transitions; }
  @Override public final ModuleRole moduleRole() { return ModuleRole.ORDERING; }  // fixed for all
}
```

Thirteen consumers extend it and cannot get their role wrong, because `moduleRole()` is `final`. Composition is preferred everywhere else: a handler *has* a repository, it is not one.

Polymorphism: one call, many behaviours.

- The bus holds a `Map<String, CommandHandler>` built from all 72 handlers. `handlers.get(command.kind()).handle(actor, command)` runs the right one with no `if` or `switch`.
- The relay calls `subscriber.on(envelope)` the same way, and the runner calls `job.run(now)`.

21. Every interface that matters

Platform extension points. Implementing one of these is how you add a feature:

Interface	Methods	Implemented by	Found by
`CommandHandler`	`kind` , `action` , `moduleRole` , `resource` , `auditsState` , `handle`	72 handler classes	`List<CommandHandler>` in `CommandBus`
`EventSubscriber<E extends DomainEvent>`	`consumerName` , `eventType` , `moduleRole` , `on`	about 80 consumers	`OutboxRelay`
`ScheduledJob`	`name` , `cron` , `moduleRole` , `run` , `onBusinessClock` , `bulkhead`	39 jobs	`ScheduledJobRunner`
`MediaTable`	a module's table of file bytes	per module	`MediaBackfillJob`

Ports that invert a dependency. Platform declares them so it never imports identity, and identity implements them:

Port (platform)	Implementation (identity)	Why a port
`CommandAuthorizer`	`PolicyDecisionPoint`	the bus must not import a business module; with no implementation it fails closed
`RequestAuthorizer`	`SessionRequestAuthorizer`	a module's web layer may not import identity
`ActorResolver`	`SessionActorResolver`	how a caller is recognised can change without the endpoint knowing
`EventPublisher`	`OutboxEventPublisher`	modules publish without knowing the outbox table

Ports to the outside world. Each port has an adapter in `infrastructure/` :

Port	Adapter	Degrade when absent
`LanguageModel`	`OpenAiCompatibleModel` (Groq, OpenRouter)	the rules' text, "rules only"
`DecisionPort`	`JevDecisions`	nothing (shadow)
`Transcription`	`GeminiTranscription`	"no transcript, play audio"
`SpeechSynthesis`	`PiperSpeech`	the phone's own voice
model serving	`ModelServingAdapter`	`DeterministicEstimator`
`StockPort` (warehouse contract)	`WarehouseStockPort`	order-level totals, said on screen
`ProofStore`	`DatabaseProofStore` , `LocalProofStore` , S3	chosen by `@ConditionalOnProperty`
`AllocationEngine`	the rules engine	swappable without touching the rules

Contract queries (`<module>/contract/*Query`), such as `OrderQuery` , `PlanQuery` , `IssueQuery` , `ReferenceQuery` and `IdentityQuery` , are the only way one module reads another. They answer with records (views) and run through `Database.readAs` as the owning module.

22. The five abstract classes

Abstract class	Extended by	What it fixes for its children
`ordering/application/OrderingConsumer<E>`	13 consumers	the `ORDERING` role, shared `OrderTransitions`
`planning/application/DraftEditHandler`	6 draft edits	find the draft at the caller's version, check the demand has not changed, store the next version; children supply only `edit(...)` and `metric()` (both `abstract`)
`issues/application/IssueCommandHandler`	6 decisions on an issue (assign, resolve, redeliver, close...)	scope check (depot or outlet, else `403`), the version guard, then the domain decides and issue, history and events commit together
`execution/application/DeliveryCommandHandler`	5 stop commands	`kind()`, `action()` and `moduleRole()` are `final`: the kind is the action, the role is Execution's, the resource is the stop
`receipt/application/ReceiptAnswerHandler`	confirm, partial receipt, dispute	find the receipt (none means nothing to confirm), the outlet's own manager only, the version guard; only the decision and the event differ

Some modules use a non-abstract base the same way (`NotificationConsumer`, `PlanningConsumer`, `IssuesConsumer`, `IdentityHandler`). Each is the **template method** pattern: the base runs the fixed steps and calls the child for the part that differs.

23. Records, enums and sealed types

Records (653). A record is an immutable class whose constructor, accessors, `equals`, `hashCode` and `toString` Java writes for you:

```
public record Actor(UUID userId, UUID deviceId) {
  public static final UUID SYSTEM_ID = UUID.fromString("00000000-0000-0000-0000-000000000001");
  public boolean isSystem() { return SYSTEM_ID.equals(userId); }
}
```

They are used for:

- every API view (`OrderView`);
- every command (`Command`) and event (`IssueRaised`);
- value objects (`Pattern` , `Choice`);
- configuration (`AiProperties`).

A **compact constructor** validates on creation. `Order` 's checks its own invariants, and `DemoSignInProperties` rejects a bad date. Immutability means a record can be shared between threads with no locks.

Enums (77). These name a closed set:

- `OrderStatus` ;
- `ErrorCode` ;
- `Effect` (ALLOW, DENY);
- `ModuleRole` (with the database role name as a field);
- `CircuitBreaker.State` (CLOSED, OPEN, HALF_OPEN).

`EnumMap` and `EnumSet` hold the order state machine's edges (section 13).

Sealed interfaces (5). A sealed interface lists every type allowed to implement it, so the compiler knows the full set:

```
public sealed interface SwitchResult permits Switched, WrongPin, Paused {}   // OperatorRegistry
```

A caller must handle all three outcomes of a PIN switch. Adding a fourth makes every non-exhaustive `switch` a compile error rather than a silent gap. The others are:

- `WarehouseReply` ;
- `StockPort.PlacementResult` ;
- `StockPort.ConfirmResult` (Confirmed, Expired, Unavailable);
- `OrphanMatcher.Match` (Found, NotFound, Ambiguous).

This replaces exceptions or `null` for expected outcomes.

Generics. `EventSubscriber<E extends DomainEvent>` ties a subscriber to its event type, so `on(EventEnvelope<IssueRaised>)` is checked at compile time. `Database.asModule(..., Supplier<T> work)` returns whatever the work returns, with its type.

Final utility classes. A domain policy such as `OrderStateMachine` , `PolicyEvaluator` , `PromptFacts` , `IssueTriage` or `ModelGate` is a `final` class with a `private` constructor and only `static` methods. They hold pure functions and no state, so they can be tested exhaustively.

24. Design patterns in the code

Pattern	Where	What it buys
Command	`Command` record + `CommandHandler` + `CommandBus`	one door for writes, idempotency, offline replay, audit
Template method	the 5 abstract bases	the fixed steps written once
Strategy	`AllocationEngine`, `AttentionScorer`, `ProofStore`	swap the algorithm or storage without touching callers
Adapter	`JevDecisions`, `OpenAiCompatibleModel`, `GeminiTranscription`, `WarehouseHttpClient`	a vendor's API shaped into our port
Ports and adapters	every `contract/` and port interface	the domain is testable and vendors are swappable
Repository	`Jdbc*Repository` (no `*Service` classes allowed)	SQL in one place per aggregate
State machine	`OrderStateMachine`, other statuses in each domain	illegal moves are impossible
Transactional outbox	`OutboxEventPublisher` + `OutboxRelay` + `ConsumerInbox`	events and state commit together; at-least-once with idempotent consumers
Circuit breaker	`intelligence/domain/CircuitBreaker`	a failing provider costs one timeout, then nothing
Bulkhead	`ScheduledJob.bulkhead()` threads	one slow dependency waits alone
Fail closed / null object	`Optional<CommandAuthorizer>` empty means deny all	missing wiring cannot open the system
Policy as data	`PolicyDocument`, `Statement`, `PolicyEvaluator`	permissions change without a deploy
Specification / rule registry	`ConstraintRegistry`	one definition of each planning rule, read by the engine, the override, the gate and the UI

25. SOLID, checked against the code

Principle	In Waypoint
Single responsibility	a handler per command kind; `CommandController` only parses and dispatches; `Database` only manages the connection identity
Open/closed	a new command, event consumer or job is a new class, with no change to the bus, relay or runner
Liskov substitution	any `LanguageModel` can replace another; every adapter returns empty rather than throwing, so callers never need to know which one they have
Interface segregation	small ports: `LanguageModel` has 3 methods, `DecisionPort` 1, `ActorResolver` 3
Dependency inversion	platform declares `CommandAuthorizer` and identity implements it; the domain declares nothing it needs from infrastructure

Part E. Module map

26. Every module, layer by layer

Each cell is the number of files in that layer, then its two largest classes.

Module	contract	domain	application	infrastructure	web
`identity`	8: IdentityQuery, ProfilePhotoQuery	8 (+ `policy/`, `oauth/`, `auth/`): McpScopes, McpReadPolicy, PolicyEvaluator	32: AccountAdminUseCase, McpOAuthHandler	6: JdbcProfilePhotos, PolicyDocumentParser	18: McpOAuthController, OperatorController
`ordering`	6: OrderViews, OrderEvents	11: Order, OrderStateMachine	19: OrderDataQuery, OrderingConsumers	1: JdbcOrderRepository	2: OrderController, AdminOrderController
`planning`	5: PlanViews, PlanEvents	33: PlanningRun, ScarceFleetReplan	30: PlanDataQuery, PlanningConsumers	9: JdbcPlanRepository, PeakDayScenario	3: PlanController, AdminPlanController
`loading`	4: LoadingViews, LoadingEvents	3: LoadingSession, ItemLine	11: LoadingViewMapper, LoadingFixture	4: JdbcLoadingRepository, JdbcManifestWriter	1: LoadingController
`execution`	4: ExecutionViews, ExecutionEvents	13: DeliveryRecord, EtaPolicy	27: ExecutionDataQuery, ProofUploads	8: JdbcDeliveryRepository, JdbcPositions	4: ExecutionController, PositionsController
`receipt`	4: ReceiptViews, ReceiptEvents	6: Handover, Receipt	7: ReceiptDataQuery, HandoverHandlers	2: JdbcReceiptRepository, JdbcHandoverRepository	1: ReceiptController
`issues`	4: IssueEvents, IssueViews	6: Issue, DecisionRules	13: IssuesConsumers, DecisionHandlers	4: JdbcIssueRepository, JdbcIssueAttachments	1: IssueController
`intelligence`	11: PredictionViews, ModelViews	29: DeterministicEstimator, ModelMetrics	29: IntelligenceDataQuery, PlanScoringJob	14: JdbcIntelligenceRepository, JdbcOutcomes	1: IntelligenceController
`messaging`	4: MessagingViews, MessagingEvents	3: MessagePolicy, SpeechText	12: MessagingConsumers, MessagingQuery	5: JdbcThreadRepository, GeminiTranscription	1: ThreadController
`notification`	3: NotificationViews, NotificationCommands	8: NotificationPolicy, Delivery	9: NotificationConsumers, NotificationHandlers	3: JdbcNotificationRepository, WebPushCrypto	1: NotificationController
`referencedata`	4: ReferenceViews, ReferenceQuery	20: ReferenceValidator, ReferenceSnapshot	19: ReferenceDataQuery, ManagedReferencePublisher	5: ReferenceVersionReader, CsvReferenceImporter	2: ReferenceController, AdminReferenceController
`warehouse`	5: StockPort, WarehouseEvents	11: PlacementDecision, Placement	11: StockUnknownRetryJob, WarehouseStockPort	9: JdbcPlacementRepository, WarehouseHttpClient	2: WarehouseController, WarehouseWebhookController
`sync`	3: SyncViews, SyncCommands	1: OperationOutcome	6: SubmitBatchHandler, OperationsQuery	1: OperationRepository	1: SyncController
`demo`	2: DemoView, DemoRuntime	2: RouteWalker, DemoSettings	7: SimulationHandlers, SimulationJob	none	1: DemoController

`platform` and `shared` are not business modules. They have no `contract/` and are organised by concern:

- `platform/db`: `Database`, `Migrator`, `ModuleRole`.
- `platform/messaging`: the bus, outbox and relay.
- `platform/scheduling`, `platform/web`, `platform/config`, `platform/audit`, `platform/storage`, `platform/observability`, `platform/time`.
- `shared`: `Actor`, `DomainException`, `ErrorCode`, `Page`, `UuidV7` and `Clock`.

Who imports whom. A module may import `shared`, `platform` and other modules' `contract` packages only. `platform` imports no business module. `shared` imports nothing internal. `ModuleBoundaryTest` fails the build otherwise.

27. The files to read first

In this order, about a day's reading:

#	File	Why
1	`AGENTS.md`	the ten rules the code obeys
2	`WaypointApplication.java`	start-up, CLI commands
3	`platform/config/DataConfig.java`	pool, `SET ROLE`, serializable
4	`platform/db/Database.java`	every transaction's identity
5	`platform/messaging/CommandHandler.java`, `CommandBus.java`, `IdempotencyGuard.java`	how every write runs
6	`platform/web/CommandController.java`, `ApiExceptionHandler.java`	the HTTP edges
7	`identity/application/PolicyDecisionPoint.java`, `identity/domain/policy/PolicyEvaluator.java`, `Pattern.java`	permissions
8	`identity/web/SessionRequestAuthorizer.java`, `SessionCookie.java`	sessions
9	`ordering/domain/Order.java`, `OrderStateMachine.java`	a pure domain
10	`ordering/application/CancelOrderHandler.java`, `ordering/infrastructure/JdbcOrderRepository.java`	a full command
11	`platform/messaging/EventPublisher.java`, `OutboxRelay.java`, `EventSubscriber.java`	events
12	`platform/scheduling/ScheduledJob.java`, `ScheduledJobRunner.java`	jobs
13	`planning/domain/ConstraintRegistry.java`, `AllocationEngine.java`, `PublicationGate.java`	the core decision
14	`intelligence/application/AiExplainer.java`, `domain/PromptFacts.java`	an AI path done safely
15	`backend/src/test/.../architecture/ModuleBoundaryTest.java`	the rules as tests

Part F. Security logic

28. Security without the Spring Security filter chain

Most Spring applications use the **Spring Security** filter chain for login, sessions and access checks. Waypoint takes only its password encoder (`spring-security-crypto`) and builds the rest itself.

Why. The requirements did not fit its defaults:

- **Policy as data**, with Allow and Deny statements on `wpt:` resources, evaluated per command, per resource and per read.
- **Two-stage checks:** authorize before the transaction and again inside it.
- **Row-level security in PostgreSQL** as the second half of every decision.
- **A shared dock device** with a PIN-switched operator.
- **MCP credentials and OAuth** for assistants.

Owning the code makes each piece explicit and unit-tested. The cost is more security code to maintain (Part G).

The whole chain, in order:

```
request
  |
OriginGuardFilter          foreign Origin on a state change -> 403          (CSRF, layer 2)
  |                        (layer 1: SameSite=Strict cookie)
McpCredentialFilter        bearer MCP token? mark the request
  |
Controller / CommandController
  |
ActorResolver / SessionRequestAuthorizer
  |   cookie "__Host-..." -> SHA-256(token) -> iam.sessions row
  |   expired (12 h absolute, 2 h idle) or disabled account -> 401
  |   MCP: client scope narrows first (never widens)
  v
PolicyDecisionPoint.denyReason(actor, action, resource)
  |   statements of the actor's roles and user attachments (cached; any change clears all)
  |   PolicyEvaluator: default deny -> any Deny wins -> any Allow -> deny
  |   denied -> 403 + audit row naming the statement
  v
CommandBus: open tx -> denyReasonInTransaction (revoked meanwhile? refuse)
  v
Database: SET LOCAL ROLE waypoint_<module>; set_config('app.actor_id', ?, true)
  v
PostgreSQL RLS: USING (app.actor_is_system() OR app.actor_has_depot(depot_code) OR ...)
  v
version guard (row_version), idempotency receipt, audit row, commit
```

The pieces, with their files:

Concern	Implementation	File
Password check	Argon2id; dummy verification for unknown emails	`identity/infrastructure/Argon2PasswordHasher.java` , `LoginHandler.java`
Brute force	throttle per account, address and identity in `iam.login_attempts`	`LoginThrottle.java` , `domain/auth/LockoutPolicy.java`
Session token	256 random bits, stored as SHA-256	`infrastructure/SessionTokens.java` , `SessionRegistry.java`
Cookie	`__Host-` , `Secure` , `HttpOnly` , `SameSite=Strict`	`web/SessionCookie.java`
CSRF	SameSite plus origin check	`platform/web/OriginGuardFilter.java`
Policy evaluation	pure; wildcards by a hand-written matcher, **not regex** (no catastrophic backtracking from user input)	`domain/policy/PolicyEvaluator.java` , `Pattern.java`
Unknown action	refused when a policy is written (the catalogue)	`PolicyAdminUseCase` , `iam.action_catalogue`
Row scope	RLS functions `app.actor_has_depot` and the others	migrations, `Database.runAs`
Least privilege in SQL	a role per module, no `DELETE` , no `BYPASSRLS` , pool role `NOINHERIT`	`DataConfig` , `ModuleRole` , migrations
Replay and double submit	`IdempotencyGuard`	`platform/messaging/`
Lost update	`row_version` guard	`Database.updateExpectingOneRow`
Audit	every command and denial	`platform/audit/AuditLog` , `AuditRedactor`
Loader PIN	5 tries, then a 5-minute pause; PBKDF2 verifier offline	`domain/PinPolicy.java` , `PinVerifier.java`

Concern	Implementation	File
Assistants	OAuth 2.1 with PKCE, client scopes, rate limits, confirmed writes	`McpOAuthHandler`, `domain/oauth/Pkce.java`, `McpRateLimiter`
Input	parameterised SQL everywhere; envelope parsed by hand, so bad input is a `422`, not a `500`	`JdbcTemplate`, `CommandController.toCommand`
Secrets	from the environment, never logged	`platform/config/*Properties`

Part G. Gaps and trade-offs

29. What this design costs, and what to watch

#	Trade-off or gap	Cost	What to watch or do
1	Custom security instead of Spring Security	more security code to own and review; no community fixes	keep the policy evaluator's tests exhaustive; review every change to `identity/web` and `platform/web` filters
2	`JdbcTemplate` returning `Map<String, Object>` in many reads	column names are strings and casts are manual (`(UUID) row.get("user_id")`); a renamed column fails at runtime, not compile time	integration tests cover each query; prefer typed row mappers in new code
3	Serializable isolation everywhere	more aborted transactions under contention; up to 5 retries each	`waypoint.db.retry` and `waypoint.db.retry.exhausted` metrics
4	A `ThreadLocal` for the ambient actor	breaks if work hops threads (`CompletableFuture` , virtual threads, reactive code)	keep commands on the request thread; pass the actor explicitly to any async work
5	Pool of 8 connections; `readAs` and `asSystemSeparately` hold a **second** connection	nested reads under load can exhaust the pool	`DatabasePoolWaiting` alert; split pools only on evidence (roadmap F-09b)
6	Scheduler, relay, circuit breakers and caches in process	per-replica state; a calendar override on one replica is unseen by another until restart	fine on one VPS; revisit before scaling out
7	362 components in one context	slower start-up and test context creation	test context caching; avoid `@MockBean` , which forks contexts
8	Java 17	no virtual threads (Java 21)	an upgrade is a planned step; check the `ThreadLocal` use first (row 4)
9	No JPA	more hand-written mapping code	the price of visible SQL, RLS and version guards
10	One `POST /api/commands` for all writes	HTTP-level tools (API gateways, OpenAPI docs) see one route	the command catalogue (`iam.action_catalogue`) and the frontend mirrors document the kinds
11	Hand-checked boundaries, no Java modules (JPMS)	the rules live in ArchUnit, not the compiler	`ModuleBoundaryTest` must run on every change; CI is currently blocked by billing, so run it by hand
12	Large classes still exist (planning's largest domain files)	harder review	split above about 300 lines, as the frontend rule asks

Part H. Practice

30. Add a command, end to end

Example: a store may "snooze" an order's reminder.

1. **Contract.** Add the kind to `ordering/contract/OrderCommands.java`, for example `SNOOZE = "order:SnoozeReminder"`, and mirror it in `frontend/src/shared/domain/ordering.ts`.
2. **Catalogue and policy (migration).** A new file `migrations/YYYYMMDDTHHMM_iam_order_snooze.sql` does three things:
   - inserts the row into `iam.action_catalogue`;
   - writes a **new default policy version** for the store manager's policy that allows it;
   - copies an existing one, such as `20261006T1001_iam_capacity_decisions.sql`.
3. **Domain.** Add `Order.snoozeReminder(Instant until)` with its rule, and a unit test with no database.
4. **Handler.** Create `ordering/application/SnoozeReminderHandler.java`:

   ```
   @Component
   public class SnoozeReminderHandler implements CommandHandler {
     public String kind() { return OrderCommands.SNOOZE; }
     public String action() { return OrderCommands.SNOOZE; }
     public ModuleRole moduleRole() { return ModuleRole.ORDERING; }
     public String resource(Command c) { return "wpt:order:order:" + CommandPayload.of(c).uuid("orderId"); }
     public Object handle(Actor actor, Command c) {
       // load at expectedVersion, call the domain, orders.update(..., expected), events.publish(...)
     }
   }
   ```

   There is nothing to register: Spring's `List<CommandHandler>` picks it up.
5. **Schema, if needed.** Each module writes its own migration, expand-only: a nullable column first.
6. **Tests.** An integration test through `CommandBus.dispatch`:
   - one success;
   - one denied-scope case (another outlet's order gives `FORBIDDEN`);
   - one stale version (`VERSION_CONFLICT`).
7. **Docs.** Add the rule to `RULES-AND-POLICIES.md`, the case and its test to `EDGE-CASES.md`, an entry in your development log, and update the `STATUS.md` row.

31. Add a read endpoint

1. Add a method to the module's query (`OrderDataQuery`) that runs in `database.readAs(ORDERING, actor.userId(), ...)` . For a list, add a `requireScope` check so that out of scope is a `403` , not an empty list.
2. Add a controller method:

```
@GetMapping("/{orderId}/reminder")
public ReminderView reminder(@PathVariable UUID orderId, HttpServletRequest request) {
  var actor = authorizer.require(request, READ, "wpt:order:order:" + orderId);
  return orders.reminder(actor, orderId);
}
```

3. Return a record from `contract/` and mirror it in `frontend/src/shared/domain/` .
4. Paginate a list with a keyset cursor (`Page<T>` , `(placed_at, id) < (?, ?)`), never `OFFSET` .
5. If the read is polled, consider adding it to `CacheablePaths` for ETags. Never add anything with personal data or credentials.

32. Run, debug and test

```
docker compose up -d db                                  # PostgreSQL in Docker
cd backend
mvn spring-boot:run                                      # serve on :8080 (reads DATABASE_URL)
java -jar target/*.jar migrate import-reference          # explicit operational commands
mvn test -Dtest=ModuleBoundaryTest                       # one test class
mvn test -Dtest='OrderingSchemaIntegrationTest#theOutletsManagerSeesTheOrderAndNoOneElseDoes'
mvn verify                                               # everything (integration needs a DB)
```

- **Integration test database:** set `TEST_DATABASE_URL`, or let Docker start a throwaway PostgreSQL. Read the report for **skipped** tests before calling a database change verified.
- **Debugging in an IDE:** run `WaypointApplication` as a Java application with `DATABASE_URL` set; breakpoints in handlers work as normal.
- **SQL and RLS:** connect with `psql`, then `SET ROLE waypoint_ordering; SELECT set_config('app.actor_id','<uuid>',false);` to see what that actor sees.
- **Metrics:** `curl localhost:8080/prometheus | grep waypoint_command`.

33. Glossary

Term	Meaning
Bean	an object Spring creates and manages
ApplicationContext	the container holding every bean
IoC / DI	the container creates dependencies and passes them in
Starter	a dependency that brings a compatible set of libraries
Auto-configuration	default beans Spring creates unless you define your own
DispatcherServlet	Spring MVC's front controller, routing requests to controllers
Filter	code that runs on every request before routing
DataSource / pool	the source of database connections (HikariCP)
JdbcTemplate	Spring's helper for running SQL safely
TransactionTemplate	programmatic transactions with chosen isolation and propagation
Isolation	how much concurrent transactions see of each other (Waypoint: serializable)
Propagation	join the current transaction or start a new one
RLS	PostgreSQL row-level security: a policy per table deciding visible rows
Port / adapter	an interface the core owns / its implementation for one vendor
Contract	a module's public views, commands, events and queries
Outbox	events written in the same transaction as the change, delivered later
Idempotency	doing the same command twice has the effect of doing it once
Optimistic concurrency	`WHERE row_version = ?`; a conflict fails instead of overwriting